Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

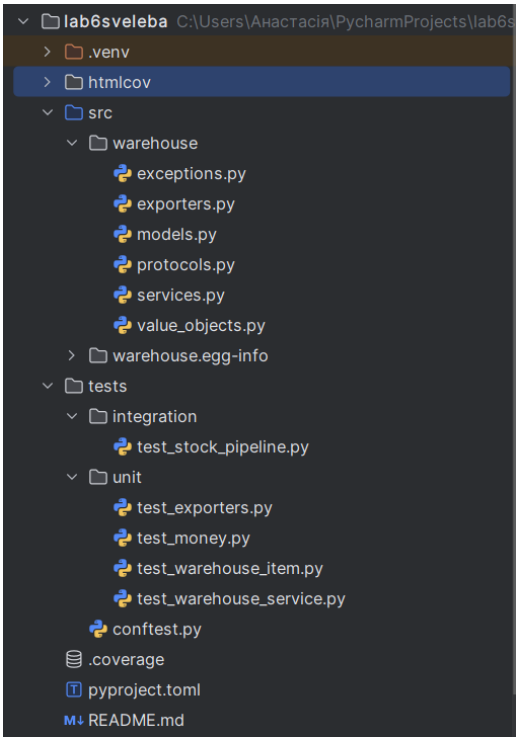
Варіант 10. Система обліку складських запасів

Протестувати:

- створення Product/WarehouseItem та валідацію quantity/price;
- операцію надходження;
- операцію списання;
- insufficient stock;
- пошук за кодом товару;
- розрахунок загальної вартості складу;
- визначення low-stock positions;
- boundary та invalid quantity/unit_price;
- CSV streaming import;
- malformed quantity record;
- JSON export;
- configuration low-stock threshold;
- WarehouseRepository через Mock/fake;
- supplier/reorder service через Mock із side_effect;
- async reorder provider через AsyncMock та integration stock flow.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/ із розділенням на unit/ та integration/.



2. У файлі pyproject.toml я описала конфігурацію проєкту, його версію, метадані збірки через setuptools, а також секцію додаткових залежностей для тестування та налаштування pytest із підтримкою покриття коду і маркерів.

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse"
version = "0.1.0"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "pytest-asyncio>=0.23.0"
]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = ["-ra", "--strict-markers"]
markers = [
    "integration: integration tests",
]

[tool.coverage.run]
branch = true
source = ["warehouse"]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

3. У файлі src/warehouse/exceptions.py я створила кастомні винятки системи (WarehouseError, InsufficientStockError, MalformedRecordError), які

забезпечують чітку обробку помилок при нестачі товару на складі чи пошкодженні даних в імпорті.

```
class WarehouseError(Exception):
    """Базовий виняток системи обліку складу."""
    pass

class InsufficientStockError(WarehouseError):
    """Виняток при нестачі товару на складі."""
    def __init__(self, message: str, available: int, requested: int) -> None:
        super().__init__(message)
        self.available = available
        self.requested = requested

class MalformedRecordError(WarehouseError):
    """Виняток для пошкоджених/некоректних записів у файлі."""
    pass
```

4. У файлі src/warehouse/value_objects.py я реалізувала незмінний (frozen=True) клас-об'єкт значення Money для роботи з грошовими сумами та валютами, додавши валідацію на заборону від'ємних значень і перевантаження оператора додавання.

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Money:
    amount: float
    currency: str = "UAH"

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError("Amount cannot be negative")

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return Money(self.amount + other.amount, self.currency)
```

5. У файлі src/warehouse/models.py я розробила головну сутність бізнес-логіки WarehouseItem із валідацією кількості, реалізувала методи надходження (receive) та списання (issuing) товарів, а також властивість обчислення загальної вартості позиції.

```
from dataclasses import dataclass
from warehouse.value_objects import Money

@dataclass
class WarehouseItem:
    product_code: str
    name: str
    quantity: int
    unit_price: Money

    def __post_init__(self) -> None:
        if self.quantity < 0:
            raise ValueError("Quantity cannot be negative")

    def receive(self, amount: int) -> None:
        if amount <= 0:
            raise ValueError("Receive amount must be positive")
```

```
        self.quantity += amount

    def issue(self, amount: int) -> None:
        if amount <= 0:
            raise ValueError("Issue amount must be positive")
        from warehouse.exceptions import InsufficientStockError
        if self.quantity < amount:
            raise InsufficientStockError(
                f"Insufficient stock for {self.product_code}",
                available=self.quantity,
                requested=amount
            )
        self.quantity -= amount

    @property
    def total_value(self) -> Money:
        return Money(self.unit_price.amount * self.quantity,
self.unit_price.currency)
```

6. У файлі src/warehouse/protocols.py я визначила інтерфейси-протоколи (WarehouseRepository, SupplierService, AsyncReorderProvider) для слабкого зв'язування компонентів і зручного тестування через мок-об'єкти.

```
from typing import Protocol, Iterator
from warehouse.models import WarehouseItem

class WarehouseRepository(Protocol):
    def get_by_code(self, code: str) -> WarehouseItem | None: ...
    def save(self, item: WarehouseItem) -> None: ...

class SupplierService(Protocol):
    def reorder(self, product_code: str, quantity: int) -> str: ...

class AsyncReorderProvider(Protocol):
    async def async_reorder(self, product_code: str, quantity: int) -> dict: ...
```

7. У файлі src/warehouse/services.py я реалізувала основний бізнес-сервіс WarehouseService, який керує надходженням та списанням запасів, обробляє винятки недостачі через виклик постачальника (side_effect), фільтрує товари з низьким залишком за змінною середовища (monkeypatch) та забезпечує потоковий CSV-імпорт з перевіркою коректності записів.

```
import os
import csv
from pathlib import Path
from typing import List, Iterator
from warehouse.models import WarehouseItem
from warehouse.value_objects import Money
from warehouse.exceptions import MalformedRecordError, InsufficientStockError
from warehouse.protocols import WarehouseRepository, SupplierService

def get_low_stock_threshold() -> int:
    return int(os.getenv("LOW_STOCK_THRESHOLD", "10"))

class WarehouseService:
    def __init__(self, repository: WarehouseRepository, supplier:
SupplierService) -> None:
        self._repo = repository
        self._supplier = supplier
```

```
    def process_incoming(self, code: str, qty: int) -> None:
        item = self._repo.get_by_code(code)
        if not item:
            raise ValueError(f"Product {code} not found")
        item.receive(qty)
        self._repo.save(item)

    def process_issuing(self, code: str, qty: int) -> None:
        item = self._repo.get_by_code(code)
        if not item:
            raise ValueError(f"Product {code} not found")
        try:
            item.issue(qty)
        except InsufficientStockError:
            # Спроба автоматичного перезамовлення через постачальника
            self._supplier.reorder(code, qty)
            raise
        self._repo.save(item)

    def get_low_stock_items(self, items: List[WarehouseItem]) ->
List[WarehouseItem]:
        threshold = get_low_stock_threshold()
        return [item for item in items if item.quantity <= threshold]

    def calculate_total_inventory_value(self, items: List[WarehouseItem]) ->
Money:
        if not items:
            return Money(0.0)
        total = sum(item.total_value.amount for item in items)
        return Money(total, items[0].unit_price.currency)

    def stream_import_csv(self, file_path: Path) -> Iterator[WarehouseItem]:
        with open(file_path, mode="r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    code = row["code"]
                    name = row["name"]
                    qty = int(row["quantity"])
                    price = float(row["unit_price"])
                    yield WarehouseItem(code, name, qty, Money(price))
                except (KeyError, ValueError) as e:
                    raise MalformedRecordError(f"Invalid CSV record: {row}")
from e
```

8. У файлі src/warehouse/exporters.py я створила функцію експорту складських даних export_inventory_json, яка серіалізує список товарів у формат JSON із збереженням кодування та форматуванням.

```
import json
from pathlib import Path
from typing import List
from warehouse.models import WarehouseItem

def export_inventory_json(items: List[WarehouseItem], path: Path) -> None:
    data = [
        {
            "code": item.product_code,
            "name": item.name,
            "quantity": item.quantity,
            "unit_price": item.unit_price.amount,
            "currency": item.unit_price.currency
        }
```

```
        for item in items
    ]
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2),
encoding="utf-8")
```

9. У файлі tests/conftest.py я налаштувала спільні для всього тестового набору фікстури (sample_item, low_stock_item, inventory_list), що забезпечують багаторазове використання тестових даних без дублювання коду.

```
import pytest
from warehouse.models import WarehouseItem
from warehouse.value_objects import Money

@pytest.fixture
def sample_item() -> WarehouseItem:
    return WarehouseItem(
        product_code="PRD-001",
        name="Laptop",
        quantity=15,
        unit_price=Money(25000.0)
    )

@pytest.fixture
def low_stock_item() -> WarehouseItem:
    return WarehouseItem(
        product_code="PRD-002",
        name="Mouse",
        quantity=3,
        unit_price=Money(500.0)
    )

@pytest.fixture
def inventory_list(sample_item, low_stock_item) -> list[WarehouseItem]:
    return [sample_item, low_stock_item]
```

10. У файлі tests/unit/test_warehouse_item.py я протестувала бізнес-операції надходження й списання товарів, граничні та невалідні значення кількості, а також перевірила правильність передачі атрибутів у кастомному винятку недостачі залишку.

```
import pytest
from warehouse.models import WarehouseItem
from warehouse.value_objects import Money
from warehouse.exceptions import InsufficientStockError

def test_item_receive(sample_item):
    sample_item.receive(5)
    assert sample item.quantity == 20

def test_item_issue(sample_item):
    sample_item.issue(5)
    assert sample_item.quantity == 10

def test_item_issue_insufficient_stock(sample_item):
    with pytest.raises(InsufficientStockError) as exc_info:
        sample_item.issue(20)
    assert exc_info.value.available == 15
    assert exc_info.value.requested == 20

@pytest.mark.parametrize("invalid_qty", [-1, 0])
def test_invalid_receive_amount(sample_item, invalid_qty):
```

```
    with pytest.raises(ValueError):
        sample_item.receive(invalid_qty)
```

11. У файлі tests/unit/test_warehouse_service.py я реалізувала юніт-тести з використанням Mock для перевірки взаємодії сервісу з репозиторієм та постачальником, перевірила side_effect при нестачі товару, конфігурацію через monkeypatch та асинхронні виклики за допомогою AsyncMock і pytest-asyncio.

```
import pytest
from unittest.mock import Mock, AsyncMock
from warehouse.services import WarehouseService, get_low_stock_threshold
from warehouse.models import WarehouseItem
from warehouse.value_objects import Money
from warehouse.exceptions import InsufficientStockError


def test_total_inventory_value(inventory_list):
    repo = Mock()
    supplier = Mock()
    service = WarehouseService(repo, supplier)

    total = service.calculate_total_inventory_value(inventory_list)
    # 15 * 25000 + 3 * 500 = 375000 + 1500 = 376500
    assert total.amount == pytest.approx(376500.0)


def test_empty_inventory_value():
    repo = Mock()
    supplier = Mock()
    service = WarehouseService(repo, supplier)
    assert service.calculate_total_inventory_value([]).amount == 0.0


def test_low_stock_filtering(inventory_list):
    repo = Mock()
    supplier = Mock()
    service = WarehouseService(repo, supplier)

    low_items = service.get_low_stock_items(inventory_list)
    assert len(low_items) == 1
    assert low_items[0].product_code == "PRD-002"


def test_checkout_insufficient_stock_side_effect(sample_item):
    repo = Mock()
    repo.get_by_code.return_value = sample_item

    supplier = Mock()
    supplier.reorder.return_value = "ORDER-999"

    service = WarehouseService(repo, supplier)

    with pytest.raises(InsufficientStockError):
        service.process_issuing("PRD-001", 100)

    # Перевірка interaction та side_effect
    supplier.reorder.assert_called_once_with("PRD-001", 100)
    repo.save.assert_not_called()


def test_configuration_monkeypatch(monkeypatch):
    monkeypatch.setenv("LOW_STOCK_THRESHOLD", "25")
```

```
    assert get_low_stock_threshold() == 25


def test_configuration_default(monkeypatch):
    monkeypatch.delenv("LOW_STOCK_THRESHOLD", raising=False)
    assert get_low_stock_threshold() == 10


@pytest.mark.asyncio
async def test_async_reorder_provider():
    provider = AsyncMock()
    provider.async_reorder.return_value = {"status": "success", "order_id":
"ORD-123"}

    result = await provider.async_reorder("PRD-001", 50)
    assert result["status"] == "success"
    provider.async_reorder.assert_awaited_once_with("PRD-001", 50)
```

12. У файлі tests/unit/test_exporters.py я перевірила коректність роботи експорту даних у JSON-формат за допомогою ізольованого тимчасового середовища файлової системи tmp_path.

```
import json
from warehouse.exporters import export_inventory_json


def test_json_export(tmp_path, inventory_list):
    output_file = tmp_path / "inventory.json"
    export_inventory_json(inventory_list, output_file)

    assert output_file.exists()
    data = json.loads(output_file.read_text(encoding="utf-8"))
    assert len(data) == 2
    assert data[0]["code"] == "PRD-001"
    assert data[0]["quantity"] == 15
```

13. У файлі tests/unit/test_money.py я написала модульні тести для перевірки коректності створення об'єктів Money, обробки винятків для від'ємних сум за допомогою pytest.raises, порівняння значень через pytest.approx та параметризованої перевірки валідних грошових номіналів.

```
import pytest
from warehouse.value_objects import Money

def test_money_creation():
    m = Money(100.0, "UAH")
    assert m.amount == 100.0
    assert m.currency == "UAH"

def test_negative_money_raises():
    with pytest.raises(ValueError, match="negative"):
        Money(-10.0)

@pytest.mark.parametrize(
    ("amount", "expected"),
    [
        (0.0, 0.0),
        (0.01, 0.01),
        (100.50, 100.50),
    ]
)
```

```
def test_valid_money_amounts(amount, expected):
    m = Money(amount)
    assert m.amount == pytest.approx(expected)

def test_money_addition():
    m1 = Money(100.0)
    m2 = Money(50.0)
    assert m1 + m2 == Money(150.0)
```

14. У файлі tests/integration/test_stock_pipeline.py я створила інтеграційний тест, який перевіряє повний конвеєр даних від потокового CSV-імпорту через тимчасові файли (tmp_path) до обробки сервісом та подальшого JSON-експорту звітів.

```
import pytest
from unittest.mock import Mock
from warehouse.services import WarehouseService
from warehouse.exporters import export_inventory_json
from warehouse.exceptions import MalformedRecordError


def test_csv_import_and_export_pipeline(tmp_path):
    csv_file = tmp_path / "stock_input.csv"
    csv_file.write_text(
        "code,name,quantity,unit_price\n"
        "PRD-100,Keyboard,10,1200.0\n"
        "PRD-101,Monitor,5,7500.0\n",
        encoding="utf-8"
    )

    repo = Mock()
    supplier = Mock()
    service = WarehouseService(repo, supplier)

    items = list(service.stream_import_csv(csv_file))
    assert len(items) == 2
    assert items[0].name == "Keyboard"

    output_json = tmp_path / "report.json"
    export_inventory_json(items, output_json)

    assert output_json.exists()


def test_csv_import_malformed_record(tmp_path):
    csv_file = tmp_path / "bad_stock.csv"
    csv_file.write_text(
        "code,name,quantity,unit_price\n"
        "PRD-100,Keyboard,invalid_qty,1200.0\n",
        encoding="utf-8"
    )
    repo = Mock()
    supplier = Mock()
    service = WarehouseService(repo, supplier)

    with pytest.raises(MalformedRecordError):
        list(service.stream_import_csv(csv_file))
```

15. У файлі README.md я описала повну документацію проєкту, включаючи опис архітектури за стандартом src-layout, інструкції з настановлення віртуального середовища та встановлення залежностей,

приклади команд для запуску тестів у деталізованому режимі, генерації звітів покриття коду за допомогою pytest-cov (включно з branch coverage) та створення інтерактивного HTML-звіту.

```
# Warehouse Inventory System & Automated Test Suite

Проєкт реалізує **Систему обліку складських запасів** (Варіант 10) та повноцінний автоматизований **Test Suite** із використанням фреймворку `pytest` відповідно до вимог Лабораторної роботи №6.

---

## Особливості системи

- **Моделі та Value Objects**:
  - `Money` — значення грошових сум із перевіркою від'ємних значень та контролем валют.
  - `WarehouseItem` — сутність товару з підтримкою операцій надходження (`receive`) та списання (`issue`).
- **Сервісна логіка**:
  - Обчислення загальної вартості складських залишків.
  - Фільтрація товарів із низьким рівнем залишків (`low stock`).
  - Потоковий CSV-імпорт (`stream_import_csv`) із валідацією записів.
  - JSON-експорт звітів.
- **Обробка винятків**:
  - `InsufficientStockError` — нестача товару при списанні.
  - `MalformedRecordError` — пошкоджені записи при імпорті.
- **Гнучке налаштування**:
  - Динамічне зчитування порогу залишків через змінну середовища `LOW_STOCK_THRESHOLD`.

---

## Структура проєкту

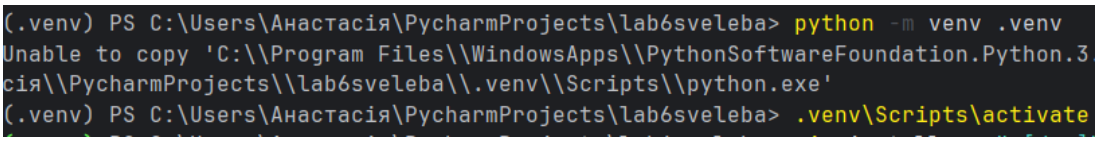
```text
warehouse_project/
├── pyproject.toml
├── README.md
├── src/
│   └── warehouse/
│       ├── __init__.py
│       ├── exceptions.py
│       ├── value_objects.py
│       ├── models.py
│       ├── services.py
│       ├── repositories.py
│       ├── exporters.py
│       └── protocols.py
└── tests/
    ├── conftest.py
    ├── unit/
    │   ├── test_money.py
    │   ├── test_warehouse_item.py
    │   ├── test_warehouse_service.py
    │   └── test_exporters.py
    └── integration/
        └── test_stock_pipeline.py
```

Інсталяція та налаштування
- **Клонування / Створення середовища**
```bash
python -m venv .venv
source .venv/bin/activate  # для Linux/macOS
```
```

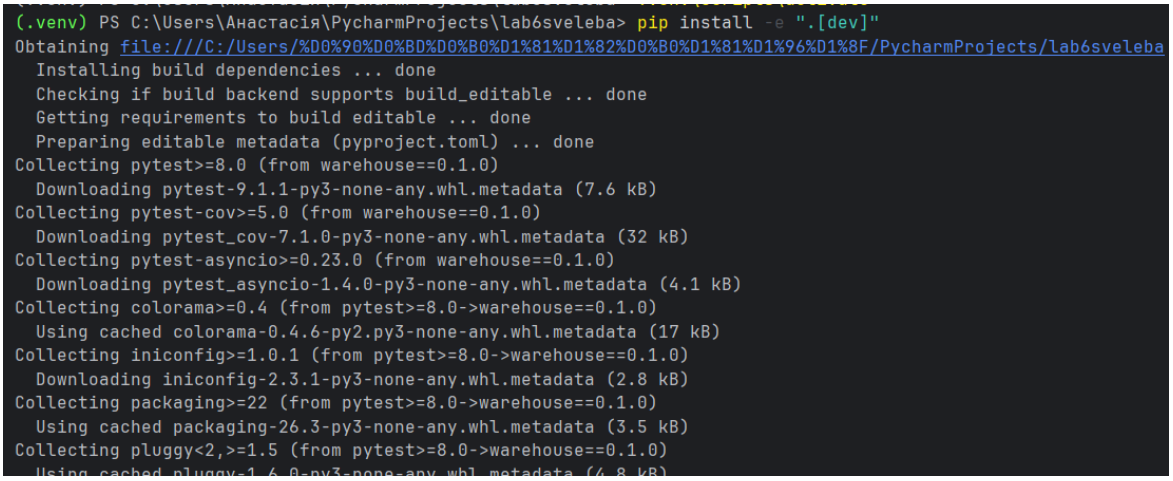
```
# або .venv\Scripts\activate для Windows
```
-**Встановлення залежностей проєкту**
```bash
pip install -e ".[dev]"
```
-**Запуск усіх тестів**
```bash
pytest
```
-**Запуск у деталізованому режимі (Verbose)**
```bash
pytest -v
```
-**Запуск із підрахунком покриття коду**
```bash
pytest -v --cov=warehouse --cov-branch --cov-report term-missing
```
-**Генерація HTML-звіту покриття**
```bash
pytest --cov=warehouse --cov-branch --cov-report=html
```
-Після виконання відкрийте htmlcov/index.html у своєму браузері.

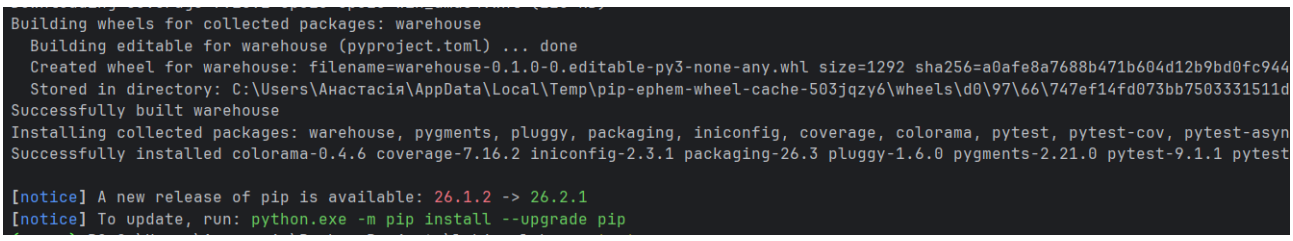
Результати:

Клонування / Створення середовища:



Встановлення залежностей проєкту:





Запуск усіх тестів:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> pytest
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 21 items

tests\integration\test_stock_pipeline.py ..                              [  9%]
tests\unit\test_exporters.py .                                           [ 14%]
tests\unit\test_money.py ......                                          [ 42%]
tests\unit\test_warehouse_item.py .....                                  [ 66%]
tests\unit\test_warehouse_service.py .......                             [100%]

============================== 21 passed in 0.09s ==============================
```

Запуск у деталізованому режимі (Verbose):

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Анастасія\PycharmProjects\lab6sveleba\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 21 items

tests/integration/test_stock_pipeline.py::test_csv_import_and_export_pipeline PASSED [  4%]
tests/integration/test_stock_pipeline.py::test_csv_import_malformed_record PASSED [  9%]
tests/unit/test_exporters.py::test_json_export PASSED                    [ 14%]
tests/unit/test_money.py::test_money_creation PASSED                     [ 19%]
tests/unit/test_money.py::test_negative_money_raises PASSED              [ 23%]
tests/unit/test_money.py::test_valid_money_amounts[0.0-0.0] PASSED       [ 28%]
tests/unit/test_money.py::test_valid_money_amounts[0.01-0.01] PASSED     [ 33%]
tests/unit/test_money.py::test_valid_money_amounts[100.5-100.5] PASSED   [ 38%]
tests/unit/test_money.py::test_money_addition PASSED                     [ 42%]
tests/unit/test_warehouse_item.py::test_item_receive PASSED              [ 47%]
tests/unit/test_warehouse_item.py::test_item_issue PASSED                [ 52%]
tests/unit/test_warehouse_item.py::test_item_issue_insufficient_stock PASSED [ 57%]
tests/unit/test_warehouse_item.py::test_invalid_receive_amount[-1] PASSED [ 61%]
tests/unit/test_warehouse_item.py::test_invalid_receive_amount[0] PASSED [ 66%]
tests/unit/test_warehouse_service.py::test_total_inventory_value PASSED  [ 71%]
tests/unit/test_warehouse_service.py::test_empty_inventory_value PASSED  [ 76%]
tests/unit/test_warehouse_service.py::test_low_stock_filtering PASSED    [ 80%]
```

```
tests/unit/test_warehouse_service.py::test_checkout_insufficient_stock_side_effect PASSED [ 85%]
tests/unit/test_warehouse_service.py::test_configuration_monkeypatch PASSED [ 90%]
tests/unit/test_warehouse_service.py::test_configuration_default PASSED  [ 95%]
tests/unit/test_warehouse_service.py::test_async_reorder_provider PASSED [100%]

============================== 21 passed in 0.09s ==============================
```

Запуск із підрахунком покриття коду:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> pytest -v --cov=warehouse --cov-branch --cov-report term-missing
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Анастасія\PycharmProjects\lab6sveleba\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 21 items

tests/integration/test_stock_pipeline.py::test_csv_import_and_export_pipeline PASSED [  4%]
tests/integration/test_stock_pipeline.py::test_csv_import_malformed_record PASSED [  9%]
tests/unit/test_exporters.py::test_json_export PASSED                    [ 14%]
tests/unit/test_money.py::test_money_creation PASSED                     [ 19%]
tests/unit/test_money.py::test_negative_money_raises PASSED              [ 23%]
tests/unit/test_money.py::test_valid_money_amounts[0.0-0.0] PASSED       [ 28%]
tests/unit/test_money.py::test_valid_money_amounts[0.01-0.01] PASSED     [ 33%]
tests/unit/test_money.py::test_valid_money_amounts[100.5-100.5] PASSED   [ 38%]
tests/unit/test_money.py::test_money_addition PASSED                     [ 42%]
tests/unit/test_warehouse_item.py::test_item_receive PASSED              [ 47%]
tests/unit/test_warehouse_item.py::test_item_issue PASSED                [ 52%]
tests/unit/test_warehouse_item.py::test_item_issue_insufficient_stock PASSED [ 57%]
tests/unit/test_warehouse_item.py::test_invalid_receive_amount[-1] PASSED [ 61%]
tests/unit/test_warehouse_item.py::test_invalid_receive_amount[0] PASSED [ 66%]
tests/unit/test_warehouse_service.py::test_total_inventory_value PASSED  [ 71%]
tests/unit/test_warehouse_service.py::test_empty_inventory_value PASSED  [ 76%]
tests/unit/test_warehouse_service.py::test_low_stock_filtering PASSED    [ 80%]
```

```
tests/unit/test_warehouse_service.py::test_checkout_insufficient_stock_side_effect PASSED [ 85%]
tests/unit/test_warehouse_service.py::test_configuration_monkeypatch PASSED  [ 90%]
tests/unit/test_warehouse_service.py::test_configuration_default PASSED      [ 95%]
tests/unit/test_warehouse_service.py::test_async_reorder_provider PASSED     [100%]

================================ tests coverage ================================
_____________ coverage: platform win32, python 3.13.14-final-0 _____________

Name                              Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------
src\warehouse\exceptions.py           9      0      0      0   100%
src\warehouse\exporters.py            7      0      0      0   100%
src\warehouse\models.py              25      2      8      2    88%   13, 22
src\warehouse\protocols.py            5      0      0      0   100%
src\warehouse\services.py            50      7      8      1    83%   19-23, 28, 35
src\warehouse\value_objects.py       12      1      4      1    88%   14
-----------------------------------------------------------------------------
TOTAL                               108     10     20      4    88%
Required test coverage of 80.0% reached. Total coverage: 87.50%
============================== 21 passed in 0.23s ==============================
```

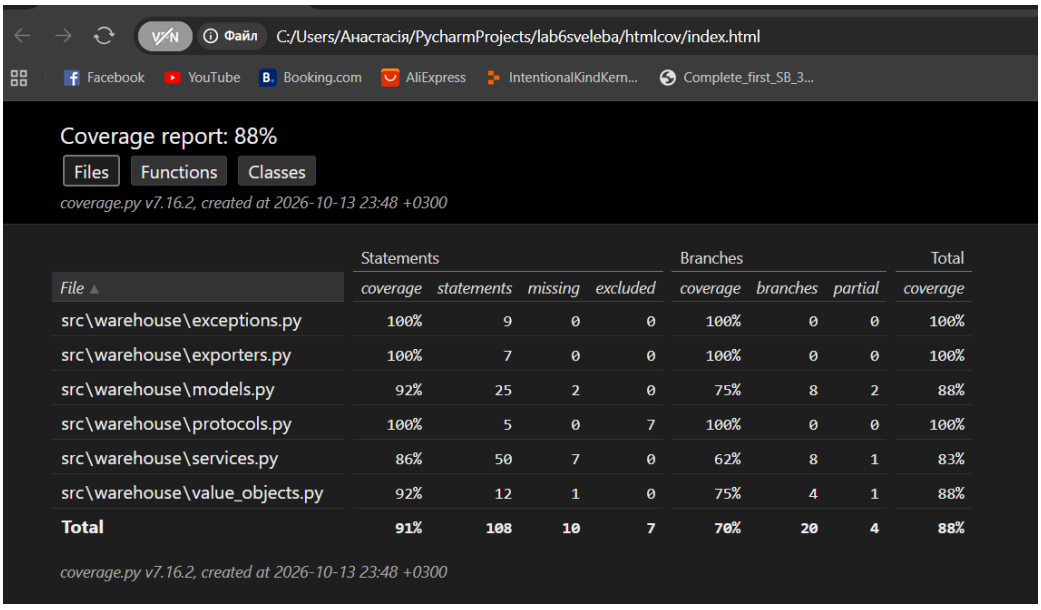
Генерація HTML-звіту покриття:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> pytest --cov=warehouse --cov-branch --cov-report=html
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 21 items

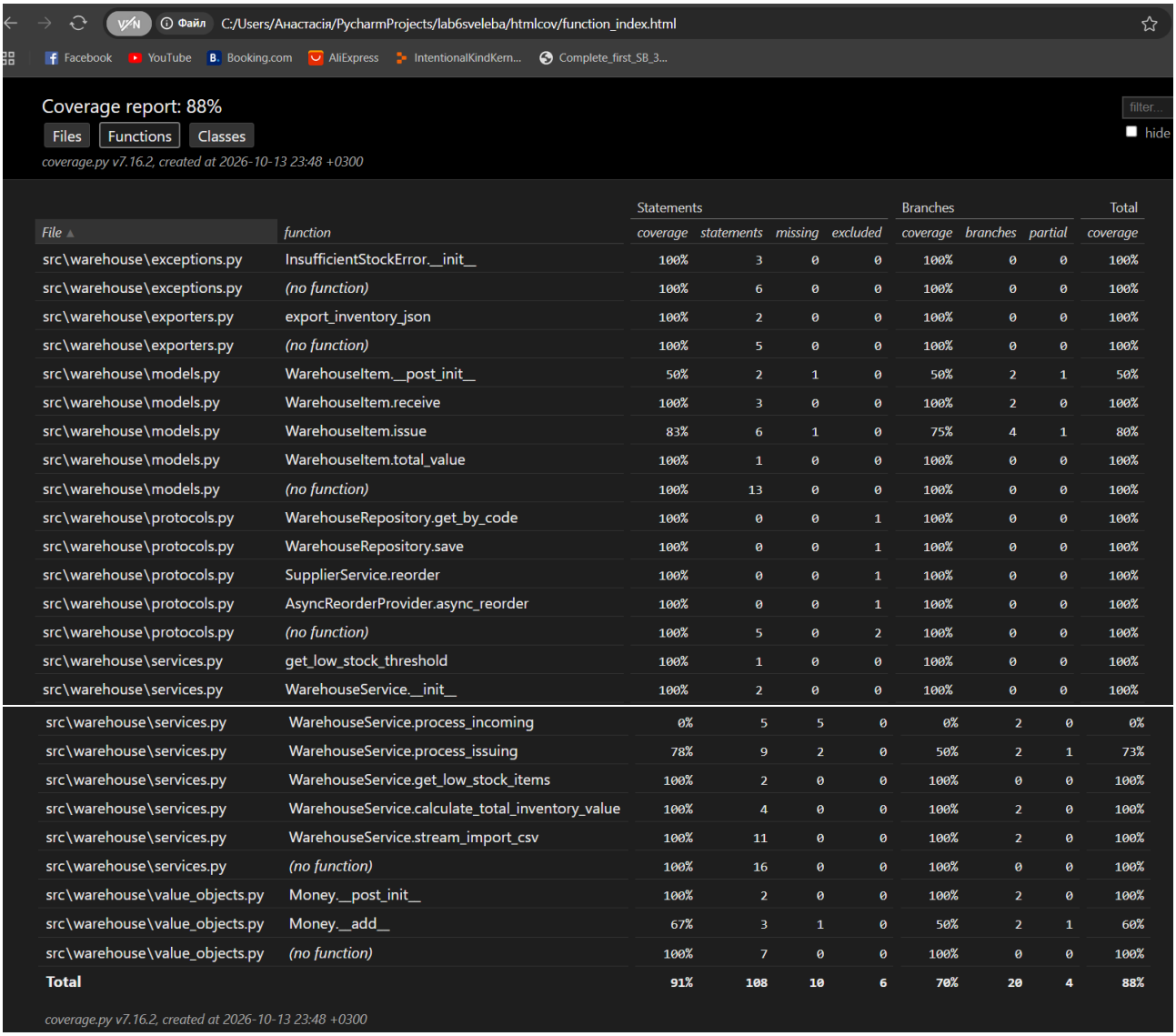
tests\integration\test_stock_pipeline.py ..                              [  9%]
tests\unit\test_exporters.py .                                           [ 14%]
tests\unit\test_money.py ......                                          [ 42%]
tests\unit\test_warehouse_item.py .....                                  [ 66%]
tests\unit\test_warehouse_service.py .......                             [100%]

================================ tests coverage ================================
_____________ coverage: platform win32, python 3.13.14-final-0 _____________

Coverage HTML written to dir htmlcov
Required test coverage of 80.0% reached. Total coverage: 87.50%
============================== 21 passed in 0.26s ==============================
```

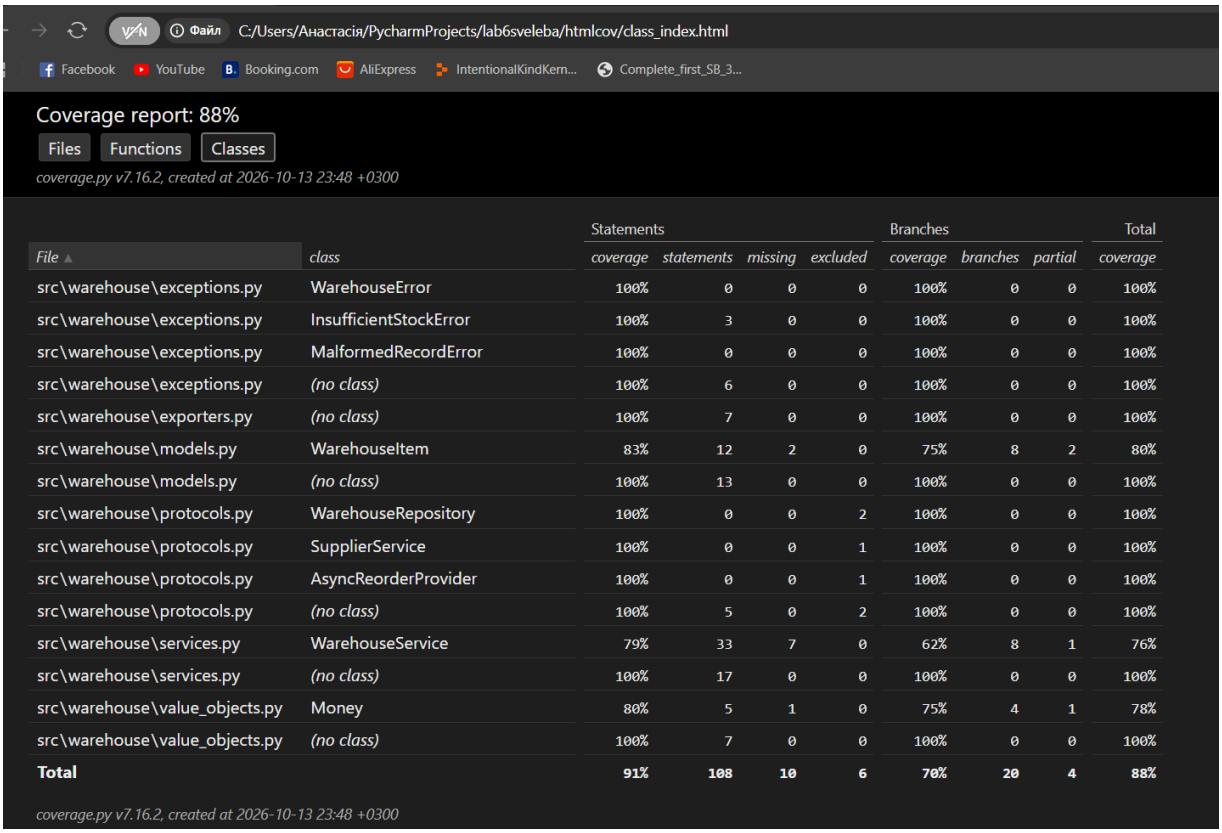


	Statements				Branches			Total
File	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse\exceptions.py	100%	9	0	0	100%	0	0	100%
src\warehouse\exporters.py	100%	7	0	0	100%	0	0	100%
src\warehouse\models.py	92%	25	2	0	75%	8	2	88%
src\warehouse\protocols.py	100%	5	0	7	100%	0	0	100%
src\warehouse\services.py	86%	50	7	0	62%	8	1	83%
src\warehouse\value_objects.py	92%	12	1	0	75%	4	1	88%
Total	**91%**	**108**	**10**	**7**	**70%**	**20**	**4**	**88%**



		Statements				Branches			Total
File ▲	function	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse\exceptions.py	InsufficientStockError.__init__	100%	3	0	0	100%	0	0	100%
src\warehouse\exceptions.py	*(no function)*	100%	6	0	0	100%	0	0	100%
src\warehouse\exporters.py	export_inventory_json	100%	2	0	0	100%	0	0	100%
src\warehouse\exporters.py	*(no function)*	100%	5	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem.__post_init__	50%	2	1	0	50%	2	1	50%
src\warehouse\models.py	WarehouseItem.receive	100%	3	0	0	100%	2	0	100%
src\warehouse\models.py	WarehouseItem.issue	83%	6	1	0	75%	4	1	80%
src\warehouse\models.py	WarehouseItem.total_value	100%	1	0	0	100%	0	0	100%
src\warehouse\models.py	*(no function)*	100%	13	0	0	100%	0	0	100%
src\warehouse\protocols.py	WarehouseRepository.get_by_code	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	WarehouseRepository.save	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	SupplierService.reorder	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	AsyncReorderProvider.async_reorder	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	*(no function)*	100%	5	0	2	100%	0	0	100%
src\warehouse\services.py	get_low_stock_threshold	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.__init__	100%	2	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.process_incoming	0%	5	5	0	0%	2	0	0%
src\warehouse\services.py	WarehouseService.process_issuing	78%	9	2	0	50%	2	1	73%
src\warehouse\services.py	WarehouseService.get_low_stock_items	100%	2	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.calculate_total_inventory_value	100%	4	0	0	100%	2	0	100%
src\warehouse\services.py	WarehouseService.stream_import_csv	100%	11	0	0	100%	2	0	100%
src\warehouse\services.py	*(no function)*	100%	16	0	0	100%	0	0	100%
src\warehouse\value_objects.py	Money.__post_init__	100%	2	0	0	100%	2	0	100%
src\warehouse\value_objects.py	Money.__add__	67%	3	1	0	50%	2	1	60%
src\warehouse\value_objects.py	*(no function)*	100%	7	0	0	100%	0	0	100%
Total		**91%**	**108**	**10**	**6**	**70%**	**20**	**4**	**88%**

coverage.py v7.16.2, created at 2026-10-13 23:48 +0300



		Statements				Branches			Total
File ▲	class	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse\exceptions.py	WarehouseError	100%	0	0	0	100%	0	0	100%
src\warehouse\exceptions.py	InsufficientStockError	100%	3	0	0	100%	0	0	100%
src\warehouse\exceptions.py	MalformedRecordError	100%	0	0	0	100%	0	0	100%
src\warehouse\exceptions.py	*(no class)*	100%	6	0	0	100%	0	0	100%
src\warehouse\exporters.py	*(no class)*	100%	7	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem	83%	12	2	0	75%	8	2	80%
src\warehouse\models.py	*(no class)*	100%	13	0	0	100%	0	0	100%
src\warehouse\protocols.py	WarehouseRepository	100%	0	0	2	100%	0	0	100%
src\warehouse\protocols.py	SupplierService	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	AsyncReorderProvider	100%	0	0	1	100%	0	0	100%
src\warehouse\protocols.py	*(no class)*	100%	5	0	2	100%	0	0	100%
src\warehouse\services.py	WarehouseService	79%	33	7	0	62%	8	1	76%
src\warehouse\services.py	*(no class)*	100%	17	0	0	100%	0	0	100%
src\warehouse\value_objects.py	Money	80%	5	1	0	75%	4	1	78%
src\warehouse\value_objects.py	*(no class)*	100%	7	0	0	100%	0	0	100%
Total		**91%**	**108**	**10**	**6**	**70%**	**20**	**4**	**88%**

coverage.py v7.16.2, created at 2026-10-13 23:48 +0300

Висновок: у результаті виконання лабораторної роботи я успішно опанувала навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest. Під час роботи було створено повноцінний тестовий набір (test suite) для системи обліку складських запасів, який охоплює модульні та інтеграційні тести, фікстури, параметризацію, перевірку винятків через pytest.raises, а також ізоляцію файлової системи за допомогою tmp_path та середовища через monkeypatch. Завдяки використанню Mock, side_effect та AsyncMock вдалося ефективно ізолювати зовнішні залежності та перевірити як сценарії успішного виконання, так і шляхи виникнення помилок. Аналіз покриття коду за допомогою pytest-cov підтвердив досягнення високого відсотка покриття за операторами та гілками (statement та branch coverage), що забезпечує надійний захист від регресій та гарантує коректність функціонування розробленого проєкту.